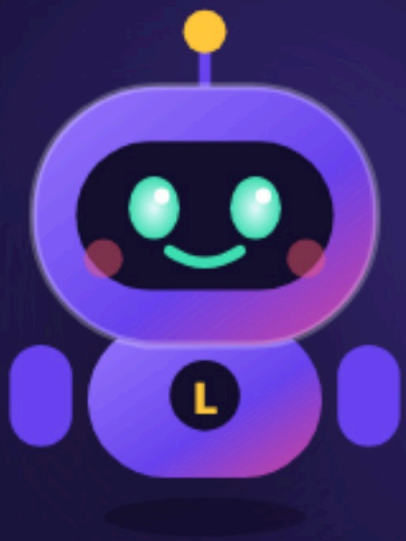
Ludoteka Campus
Colegio Francisco Arango

PROTECCIÓN DE LA PLATAFORMA

Manual de Seguridad

Autenticación, sesiones, autorización, protección web, biometría, datos personales y riesgos conocidos.



EDICIÓN

Versión 1.0 · octubre de 2026

EQUIPO

Darwin Joan Aveiga Mora · Arquitectura y desarrollo full stack
Mariana Sofía Azabache Moreno · Project manager, diseño UI/UX y frontend
Julián Esteban Chávez Zamora · QA y apoyo en backend

Contenido

01 Resumen
02 Autenticación
03 Sesiones
04 Autorización
05 Protección web
06 Auditoría
07 IA y privacidad
08 Integridad en exámenes
09 Datos personales (Ley 1581)
10 Secretos
11 Riesgos conocidos

Cómo se protege la plataforma, qué valores usa y dónde está cada control en el código. La última sección lista los riesgos conocidos que aún no están resueltos.

Resumen

Capa	Control principal
Identidad	Contraseña argon2id + código de 6 dígitos por correo; dispositivo de confianza; passkeys; acceso facial con consentimiento
Sesión	JWT de acceso de 20 min en memoria de la pestaña; refresh opaco rotativo en cookie `HttpOnly` con detección de reuso; una cuenta por navegador
Autorización	Guardias globales con `@Roles` en cada ruta y filtros por relación (docente–curso, acudiente–hijo) en los servicios
Web	CSP estricta sin scripts en línea, HSTS, `frame-ancestors 'none'`, límite de intentos en rutas de ingreso
Datos	TLS hacia CockroachDB, secretos solo en variables de entorno, plantillas faciales con AES-256-GCM, exportación y solicitudes de datos (Ley 1581)
IA	Seudónimos y redacción por función, consentimiento del acudiente para el tutor, SQL del Observatorio en caja de arena

Autenticación

Contraseña y código por correo

`modules/identity/identity.service.ts`

1. Se verifica la contraseña con argon2id (parámetros por defecto de la librería). Correo desconocido y contraseña errada responden igual: `401 INVALID_CREDENTIALS`.
2. Si la cuenta está bloqueada responde `429 ACCOUNT_LOCKED`; si está desactivada, `403 ACCOUNT_DISABLED`.
3. Si este navegador ya tiene la sesión abierta de **otra** cuenta, responde `409 SESSION_ACTIVE` (ver Una cuenta por navegador).
4. Si el navegador es de confianza para esa cuenta, abre la sesión. Si no, envía un código.

Código de un solo uso:

- 6 dígitos con `crypto.randomInt`; vence en 10 minutos; 5 intentos; reenvío cada 30 s como mínimo.
- En la base solo queda `HMAC-SHA256(JWT_SECRET, "otp:<reto>:<código>")`; la comparación es de tiempo constante.
- Pedir un código nuevo invalida los anteriores; cada código se consume una sola vez con una actualización condicional.
- El correo se guarda redactado en `email_outbox`. Fuera de producción la respuesta incluye `devCode` para pruebas.

Política de contraseña: mínimo 8 caracteres con al menos una letra y un número (máximo 128). Las cuentas creadas por el administrador nacen con clave temporal y `must_change_password`; mientras no la cambien, toda la API responde `403 PASSWORD_CHANGE_REQUIRED` salvo las rutas marcadas con `@AllowPendingPassword`.

Bloqueo progresivo: tras 5 fallos seguidos la cuenta se bloquea 1, 2, 4, 8, 16, 32 y luego 60 minutos. Se cuenta por cuenta, no por IP, y se reinicia con un ingreso correcto o un cambio de clave.

Recuperación: token aleatorio guardado como SHA-256, válido 1 hora. Cambiar o restablecer la clave revoca todas las sesiones.

Dispositivo de confianza

- Al validar el código, el usuario puede marcar «Recordar este dispositivo» por 30 días.
- La cookie `lk_device` (`HttpOnly`, `SameSite=Lax`, `Secure` en producción, `path=/api/auth`) guarda hasta 8 tokens, uno por cuenta, para equipos compartidos.
- La base guarda solo el SHA-256 del token, ligado al usuario. Solo evita el código: la contraseña se sigue pidiendo.
- Se revocan desde Ajustes → Seguridad, con «Cerrar todas las sesiones» o automáticamente si se detecta reuso de un refresh.

Passkeys (Face ID, Touch ID, huella, Windows Hello)

`modules/identity/passkeys.service.ts` con `@simplewebauthn/server`.

- `rpID` es el host de `APP_PUBLIC_URL`; los orígenes aceptados son ese y los de `CORS_ORIGIN` con el mismo host.
- Credenciales residentes, verificación de usuario obligatoria, sin atestación, máximo 10 por cuenta.
- Retos guardados en `webauthn_challenges`, de un solo uso y con 5 minutos de vigencia. El contador de firma se actualiza en cada ingreso.
- **Solo se registran desde una sesión verificada** (`amr` igual a `otp`, `trusted_device` o `passkey`): una sesión abierta solo con rostro no puede agregar llaves (`403 STEP_UP_REQUIRED`).
- El ingreso con passkey respeta la cuenta desactivada y el bloqueo por contraseña.

Acceso facial

`apps/web/src/shared/face.ts` y `modules/biometrics`.

- **Las fotos no salen del navegador.** face-api calcula un descriptor de 128 números. La API rechaza cualquier cuerpo con campos de imagen (`rejectImageFields`) y valida que cada valor esté en [-2, 2].
- **Registro:** entre 3 y 10 muestras desde una sesión verificada; se guarda el promedio si la dispersión no supera 0,42.
- **Cifrado:** AES-256-GCM con IV aleatorio de 12 bytes y el id del usuario como dato autenticado, para que una plantilla no se pueda copiar a otra fila. La llave es `SHA-256(FACIAL_ENC_KEY)` o, si está vacía, se deriva de `JWT_SECRET`. **Debe ser la misma en todos los entornos que comparten la base**; si cambia, ninguna plantilla se puede descifrar.
- **Comparación:** distancia euclidiana ≤ `FACIAL_MAX_DISTANCE` (0,5). Sin correo (identificación) exige además 0,08 de ventaja sobre el segundo candidato.
- **Bloqueo:** 5 fallos en 15 minutos, por usuario o por IP en modo identificación (`423 FACE_LOCKED`).

- **Consentimiento:** el usuario acepta la política vigente (`FACIAL_POLICY_VERSION`). Un menor necesita además la autorización de su acudiente; si el acudiente la revoca, la plantilla se borra.
- **Prueba de vida:** el servidor elige el gesto (parpadear o girar) y emite un reto de 90 s; el gesto se comprueba en el navegador (ver Riesgos conocidos).
- Cada intento queda en `face_attempts` con el correo como SHA-256.

Sesiones

Pieza	Detalle
Token de acceso	JWT firmado con `JWT_SECRET` , 20 minutos. Lleva `sub` , `role` , `email` , `fullName` , `isMinor` , `amr` (cómo se abrió la sesión) y `mcp` si falta cambiar la clave. Se envía en `Authorization: Bearer`
Dónde vive en la web	Variable del módulo `shared/api.ts` y `sessionStorage['eq_access']` : muere al cerrar la pestaña y no es accesible desde otro origen
Refresh	32 bytes aleatorios en la cookie `eq_refresh` (`HttpOnly` , `SameSite=Lax` , `Secure` en producción, `path=/api/auth` , 14 días). La base guarda el SHA-256
Rotación	Cada `/api/auth/refresh` revoca el token y entrega uno nuevo de la misma familia
Reuso	Si llega un refresh ya rotado: dentro de 30 s se trata como carrera entre pestañas (`SESSION_RACE` , la web reintenta); después se revocan **todas** las sesiones y dispositivos de confianza del usuario y queda `session.reuse_detected` en la auditoría
Cierre	`logout` revoca la familia de la cookie; `logout-all` revoca todo. Desde Ajustes se cierra una sesión o se olvida un dispositivo puntual

El JWT de acceso no se puede revocar: sigue siendo válido hasta 20 minutos después de cerrar sesión. Por eso es corto y no se guarda en almacenamiento persistente.

Una cuenta por navegador

Todas las pestañas comparten la cookie `eq_refresh` , así que un segundo ingreso con otra cuenta reemplazaría en silencio la sesión que las demás pestañas siguen mostrando. Para evitarlo:

- **API:** `IdentityService.assertBrowserAccount` rechaza con `409 SESSION_ACTIVE` el ingreso con contraseña o código si la cookie pertenece a una sesión vigente de otra cuenta. En passkey y rostro, que solo revelan la cuenta después de verificar, `keepBrowserAccount` cierra la sesión recién creada y devuelve el mismo error. Volver a entrar con la misma cuenta sí está permitido.
- **Web:** `/ingresar` y `/ingresar/rostro` redirigen a `/app` si ya hay sesión, y la portada cambia «Ingresar» por «Ir a mi campus».
- **Entre pestañas:** `shared/auth.tsx` usa `BroadcastChannel('eq-session')` . Cerrar sesión en una pestaña la cierra en todas; un ingreso en otra pestaña actualiza las que estaban sin sesión.

Para usar otra cuenta hay que cerrar la sesión actual.

Autorización

En la API

`core/auth/guards.ts`, registrados como guardias globales:

1. **`JwtAuthGuard`**: toda ruta exige token salvo las marcadas `@Public()`. Bloquea con `PASSWORD_CHANGE_REQUIRED` si el token trae `mcp`.
2. **`RolesGuard`**: aplica `@Roles(...)`. Sin `@Roles`, basta estar autenticado.

El rol abre la puerta; la relación decide qué datos se ven. Ejemplos:

Recurso	Regla	Archivo
Curso	Administrador: todo. Coordinación y rectoría: solo lectura (`403 READ_ONLY` al escribir). Docente: solo los suyos. Estudiante: solo publicados de su grupo y año	`learning/learning.service.ts` (`assertAccess`)
Boletín y resumen de un estudiante	El propio estudiante, la dirección, el docente que le dicta y el acudiente con vínculo activo	`analytics/analytics.service.ts` (`assertStudentAccess`)
Portal de familia	Solo hijos con `guardian_links` activo; los mensajes exigen además que el docente le dicte al estudiante	`governance/family.service.ts`
Integridad	El docente ve solo sus cursos; el estudiante solo reporta eventos de su propio intento	`integrity/integrity.service.ts`
Observatorio	Docente limitado a sus grupos; sin grupos, `403 NO_GROUPS`	`analytics/observatory.service.ts`

En la web

`apps/web/src/app/nav.ts` arma `ROUTE_ROLES` desde el mapa de cada rol y `RoleRoute` muestra «Esta sección no es para tu rol». Es solo experiencia de usuario: la seguridad la pone la API. La tabla completa está en Roles y permisos.

Protección web

Cabeceras (`core/security/headers.ts`, helmet):

- **CSP** sin valores por defecto:
  - `script-src 'self'`: sin scripts en línea ni `eval`.
  - `style-src` permite `'unsafe-inline'` y Google Fonts.
  - `img-src` y `media-src` permiten Cloudinary.
  - `connect-src` permite el propio host por `ws(s)` y la API de Cloudinary.
  - `frame-src` solo Cloudinary y `youtube-nocookie`.
  - `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-ancestors 'none'`.

- **HSTS** de un año con subdominios, solo si `APP_PUBLIC_URL` es HTTPS.
- `Referrer-Policy: strict-origin-when-cross-origin`, `X-Frame-Options: DENY`, `Cross-Origin-Resource-Policy: same-origin` (salvo `/email`, que lo leen los clientes de correo).
- `Permissions-Policy`: cámara, micrófono, pantalla completa y WebAuthn solo para el propio origen; geolocalización, pagos, USB, serial y Bluetooth apagados.

Límite de intentos (`core/security/throttle.ts`, `AuthThrottleGuard`):

- 300 peticiones por IP cada 15 minutos.
- 10 por IP y sujeto (correo o reto) cada 5 minutos.
- Se aplica a ingreso, código, cambio y recuperación de clave, passkeys y rostro. Responde `429 TOO_MANY_REQUESTS` con `retryAfter`.

Otros:

- CORS con lista explícita y credenciales.
- Validación con zod por ruta (`ZodPipe`): `400 VALIDATION` con el primer error.
- El filtro de errores nunca devuelve trazas: lo desconocido sale como `500 INTERNAL`.
- El build de la web no publica mapas de código y ofusca los chunks propios. Esto solo disuade; no es un control de seguridad.

Auditoría

`audit_log` registra quién hizo qué sobre qué entidad, con metadatos en JSON:

Área	Acciones
Ingreso y sesiones	`login` (con método), `login.failed`, `login.locked`, `session.reuse_detected`, `session.revoke_all`, `session.revoke`, `device.forget`
Credenciales	`password.change`, `password.reset`, `passkey.register`, `passkey.remove`
Biometría	`facial.consent`, `facial.enroll`, `facial.delete` (los intentos van a `face_attempts`)
Acudientes	`guardian.consent`, `guardian.grant`, `guardian.revoke`, `guardian.link`, `guardian.unlink`
Datos personales	`legal.accept`, `legal.accept_for_minor`, `data.export`, `data.delete_request`, `data.delete_done`, `data.delete_rejected`
Administración	`user.create`, `user.update`, `user.disable`, `user.enable`, `user.resend_access`, `user.force_password`, `users.import`, `structure.import`
Contenido y firmas	`news.create`, `news.delete`, `library.create`, `library.delete`, `signature.<estado>`

IA y privacidad

`AiService` envía texto al proveedor sin identificadores de usuario (el id solo va a `ai_usage`). La protección de datos personales la hace cada función antes de llamar:

Función	Qué se envía	Protección
Tutor	Texto de la lección, extractos de la biblioteca y los últimos 12 mensajes	El estudiante menor necesita la autorización `ai_tutor` del acudiente; se bloquea durante un examen activo
Recomendaciones al docente	Promedio y nombre del curso	El estudiante va como «el estudiante»
Resumen de foro	Hilo completo	Seudónimos («Estudiante N»), nombres del curso reemplazados, correos y teléfonos quitados; los nombres se restauran en la respuesta
Resumen del Cuaderno	Notas del estudiante	Se quitan correos, URLs y números de 7 o más dígitos
Mi ruta	Tema y nivel	Sin datos personales; el servidor recalcula la respuesta correcta
Observatorio	Pregunta en español	Los nombres de estudiantes se cambian por `ESTUDIANTE_n` antes de enviar

Observatorio (texto a SQL): dos barreras independientes.

1. **Validador estático** (`analytics/observatory.sql.ts`):
   - una sola sentencia `SELECT` o `WITH` de hasta 4000 caracteres;
   - sin comentarios, `$`, barras invertidas ni nombres calificados;
   - lista de palabras y funciones prohibidas (escritura, `pg_sleep`, catálogos del sistema, `crdb_internal` …);
   - solo las 12 vistas `v_*`;
   - `LIMIT` forzado a 201 como máximo.
2. **Base de datos:**
   - transacción `READ ONLY` con `statement_timeout` de 5 s, `SET LOCAL ROLE ludoteka_reader` (solo `SELECT` sobre las vistas) y `ROLLBACK` siempre;
   - para docentes, cada vista se reemplaza por una CTE filtrada a sus grupos.

Integridad en exámenes

- El navegador del estudiante reporta salida de pantalla completa, pestaña oculta, pérdida de foco y Escape.
- Cada evento se guarda y suma en `attempt_flags`. Al llegar a `INTEGRITY_SUSPEND_AFTER` (3), el intento se suspende y se entrega.
- En prácticas no cuenta.

- Mientras haya un examen o sala en vivo activa, el tutor IA no responde.

Datos personales (Ley 1581)

- **Documentos versionados** (términos, tratamiento de datos, cookies) en `legal_documents`. Publicar una versión nueva obliga a todos a aceptarla de nuevo.
- **Menores:** si tienen acudiente vinculado, el acudiente acepta por ellos (`legal.accept_for_minor`). Sin acudiente, el menor declara que lo leyó acompañado.
- **Autorizaciones del acudiente:** `facial` , `ai_tutor` y `data_processing` , versionadas y de solo inserción (la última decisión manda).
- **Exportación** (`GET /api/me/export`): todas las filas del usuario en las tablas con `user_id` , `student_id` , `guardian_id` o `author_id` , sin columnas de hash, token, llave ni vector, y sin tablas de credenciales.
- **Solicitud de eliminación:** una pendiente a la vez; la resuelve el administrador.
- **Verificación de documentos:** los reportes llevan un código público que se consulta en `/verificar-reporte/<código>` ; la respuesta muestra solo metadatos y el primer nombre de quien lo generó. Los certificados se verifican en `/verificar/<código>` .

Secretos

- Viven en `apps/api/.env` (ignorado por Git; solo `.env.example` está versionado) y en el panel de Render (`sync: false` en `render.yaml`).
- Son secretos: `DATABASE_URL` , `JWT_SECRET` , `AI_API_KEY` , `CLOUDINARY_URL` , `RESEND_API_KEY` , `FACIAL_ENC_KEY` y `SEED_PASSWORD` .
- `JWT_REFRESH_SECRET` existe por compatibilidad pero no se usa: el refresh es opaco.
- `JWT_SECRET` **hace tres cosas:** firma los JWT, firma los códigos por correo y, si `FACIAL_ENC_KEY` está vacía, deriva la llave facial. Rotarlo cierra todas las sesiones y vuelve ilegibles las plantillas faciales. Antes de rotarlo, define `FACIAL_ENC_KEY` con la llave actual (ver Operación).
- Nunca se imprimen valores de `.env` en registros, issues ni chats. Si un secreto se expone, se rota.

Riesgos conocidos

Hallazgos de la revisión del código que siguen abiertos, ordenados por prioridad.

#	Riesgo	Impacto	Mitigación propuesta
1	La aceptación de documentos legales solo se exige en la web (`Gate`); la API no bloquea a quien la tiene pendiente	Un cliente directo a la API omite la aceptación	Guardia global que responda `LEGAL_PENDING` salvo en `/auth` , `/legal` y `/me`

#	Riesgo	Impacto	Mitigación propuesta
2	La solicitud de eliminación solo cambia de estado; ningún código borra los datos	Incumplimiento si se marca «hecha» sin borrar a mano	Proceso de borrado o anonimización al resolverla
3	La prueba de vida facial se verifica solo en el navegador; el servidor valida que el reto exista y no se haya usado	Un cliente modificado podría enviar un descriptor sin hacer el gesto	Enviar al servidor la secuencia de descriptores del gesto y verificar la variación
4	`FACIAL_RETENTION_DAYS` no se aplica: `purgeExpired()` no se llama desde ningún lado	Plantillas sin vencimiento	Llamarlo desde la tarea periódica de Governance o desde la CLI
5	`live:join` une cualquier socket a una sala en vivo sin verificar que el usuario esté inscrito	Un tercero con el código ve las preguntas en tiempo real	Exigir token y participación antes de unir
6	Los códigos de reporte son secuenciales y la verificación pública no tiene límite de intentos	Se puede enumerar el archivo de reportes (solo metadatos)	Agregar `AuthThrottleGuard` y un sufijo aleatorio al código
7	La autorización `data_processing` se registra pero no se consulta en ninguna parte	El consentimiento no tiene efecto	Definir qué tratamiento depende de ella y exigirla
8	El tutor envía al proveedor el texto del estudiante sin redactar	Datos que el estudiante escriba llegan a Gemini	Aplicar la misma redacción del Cuaderno antes de enviar
9	El acceso facial no respeta el bloqueo por contraseña (`locked_until`)	Una cuenta bloqueada puede entrar con rostro	Revisar `locked_until` en `BiometricsService.login`
10	El Observatorio devuelve el texto del error de la base al cliente	Filtra detalles del esquema a quien ya puede consultarlo	Mensaje genérico y detalle solo en el registro
11	`FACIAL_ENC_KEY` no está en `render.yaml` ; producción deriva la llave de `JWT_SECRET`	Rotar `JWT_SECRET` destruye las plantillas	Definir `FACIAL_ENC_KEY` explícita en todos los entornos
12	`SEED_PASSWORD` tiene un valor por defecto escrito en el código, y las cuentas de laboratorio (incluida una de administración) se recrean en cada arranque si faltan	Una instalación sin esa variable deja cuentas con una clave pública; el código por correo es la única barrera	Quitar el valor por defecto y no arrancar en producción sin `SEED_PASSWORD` ; desactivar (no borrar) las cuentas de laboratorio que no se usen